

XpyQ Linear Algebra SDK README

This guide explains how to install the SDK, run all 20 supported API calls, and test three practical use cases using the bundled Python client. The SDK uses a hidden internal `config.py` for the deployment URL, while users normally work only with `la_client.py`, `test_app.py`, and `requirements.txt`.[\[cite:487\]](#)[\[cite:450\]](#)

Files in the bundle

- `config.py` — internal base URL configuration used by the client import path.
- `la_client.py` — Python SDK with 20 callable methods.
- `test_app.py` — simple smoke-test example.
- `requirements.txt` — dependency list for pip installation.[\[cite:498\]](#)[\[cite:450\]](#)

Setup

1. Extract the bundle to a folder such as `internal_config_sdk`.
2. Create and activate a Python virtual environment.
3. Install dependencies from `requirements.txt`.
4. Put a valid API key into `test_app.py` or your own runner script.

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

A `requirements.txt` file is the standard plain-text dependency format used by pip, and `pip install -r requirements.txt` installs the listed packages into the active environment.[\[cite:498\]](#)[\[cite:450\]](#)

Connection model

The SDK is built on Python Requests, which supports JSON request bodies, HTTP headers, and explicit timeout control for API calls.[\[cite:497\]](#)[\[cite:499\]](#)

The underlying service appears to be a FastAPI application or a similar OpenAPI-based API, because services of that style commonly expose `/docs` interactive documentation and `/openapi.json` schemas that the SDK already references through helper methods.[\[cite:487\]](#)[\[cite:488\]](#)

Minimal test

Edit `test_app.py` and replace:

```
API_KEY = "PUT_REAL_API_KEY_HERE"
```

Then run:

```
python test_app.py
```

FastAPI commonly exposes automatic documentation at `/docs`, which is useful for checking endpoint availability alongside the SDK methods when you need to inspect request and response structure.^[cite:487]

All 20 SDK calls

The SDK exposes 20 primary linear algebra methods. The list below maps each method to its intended operation.

Method	Purpose
<code>matmul(a, b)</code>	Matrix multiplication
<code>inv(a)</code>	Matrix inverse
<code>det(a)</code>	Determinant
<code>eig(a)</code>	Eigenvalues and eigenvectors
<code>svd(a)</code>	Singular value decomposition
<code>solve(a, b)</code>	Solve linear system
<code>transpose(a)</code>	Matrix transpose
<code>add(a, b)</code>	Elementwise matrix addition
<code>sub(a, b)</code>	Elementwise matrix subtraction
<code>hadamard(a, b)</code>	Elementwise multiplication
<code>trace(a)</code>	Matrix trace
<code>norm(a)</code>	Matrix norm
<code>rank(a)</code>	Matrix rank
<code>qr(a)</code>	QR decomposition
<code>cholesky(a)</code>	Cholesky decomposition
<code>pinv(a)</code>	Pseudoinverse
<code>reshape(a, rows, cols)</code>	Reshape matrix
<code>concat(a, b, axis)</code>	Concatenate matrices

Method	Purpose
<code>diag(values)</code>	Build diagonal matrix
<code>scalar_mul(a, scalar)</code>	Multiply matrix by scalar

Example runner for all 20 calls

Create a file named `run_all_calls.py` in the same folder:

```
from la_client import LinearAlgebraClient, LinearAlgebraAPIError

API_KEY = "PUT_REAL_API_KEY_HERE"

A = [[4, 7], [2, 6]]
B = [[1, 2], [3, 4]]
C = [[5, 6], [7, 8]]
SPD = [[4, 1], [1, 3]]
RECT = [[1, 2, 3], [4, 5, 6]]
V = [10, 20, 30]
RHS = [[1], [0]]

try:
    with LinearAlgebraClient(api_key=API_KEY) as client:
        print("1 matmul", client.matmul(B, C))
        print("2 inv", client.inv(A))
        print("3 det", client.det(A))
        print("4 eig", client.eig(A))
        print("5 svd", client.svd(RECT))
        print("6 solve", client.solve(A, RHS))
        print("7 transpose", client.transpose(RECT))
        print("8 add", client.add(B, C))
        print("9 sub", client.sub(C, B))
        print("10 hadamard", client.hadamard(B, C))
        print("11 trace", client.trace(A))
        print("12 norm", client.norm(A))
        print("13 rank", client.rank([[1, 2], [2, 4]]))
        print("14 qr", client.qr(RECT))
        print("15 cholesky", client.cholesky(SPD))
        print("16 pinv", client.pinv(RECT))
        print("17 reshape", client.reshape([[1, 2], [3, 4]], 1, 4))
        print("18 concat axis0", client.concat([[1, 2]], [[3, 4]], axis=0))
        print("19 diag", client.diag(V))
        print("20 scalar_mul", client.scalar_mul(B, 2.5))
except LinearAlgebraAPIError as e:
    print("API error:", e.status_code, e.message, e.payload)
except Exception as e:
    print("Unexpected error:", str(e))
```

Python Requests supports both JSON POST bodies and timeout arguments, which is why the SDK can wrap each matrix operation in a straightforward request method with defensive timeout handling.[cite:497][cite:499]

Run it with:

```
python run_all_calls.py
```

Three use cases

Use case 1: Portfolio covariance workflow

This example uses inverse, matrix multiplication, trace, norm, and scalar multiplication to support portfolio-risk style experiments.

```
from la_client import LinearAlgebraClient

API_KEY = "PUT_REAL_API_KEY_HERE"

cov = [
    [0.10, 0.02, 0.01],
    [0.02, 0.08, 0.03],
    [0.01, 0.03, 0.09],
]
weights = [[0.5, 0.3, 0.2]]

with LinearAlgebraClient(api_key=API_KEY) as client:
    cov_inv = client.inv(cov)
    weighted = client.matmul(weights, cov)
    risk_trace = client.trace(cov)
    risk_norm = client.norm(cov)
    scaled = client.scalar_mul(cov, 0.5)

    print("cov_inv", cov_inv)
    print("weighted", weighted)
    print("risk_trace", risk_trace)
    print("risk_norm", risk_norm)
    print("scaled", scaled)
```

Use case 2: Solving a linear system

This example uses solve, det, rank, and transpose to inspect and solve a system of equations.

```
from la_client import LinearAlgebraClient

API_KEY = "PUT_REAL_API_KEY_HERE"
A = [[3, 1], [1, 2]]
b = [[9], [8]]

with LinearAlgebraClient(api_key=API_KEY) as client:
    x = client.solve(A, b)
    determinant = client.det(A)
    matrix_rank = client.rank(A)
    at = client.transpose(A)

    print("solution", x)
    print("determinant", determinant)
```

```
print("rank", matrix_rank)
print("transpose", at)
```

Use case 3: Dimensionality reduction and factor analysis

This example uses `svd`, `qr`, `pinv`, `diag`, `reshape`, and `concat` on rectangular data matrices.

```
from la_client import LinearAlgebraClient

API_KEY = "PUT_REAL_API_KEY_HERE"
X = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 10],
]

with LinearAlgebraClient(api_key=API_KEY) as client:
    svd_out = client.svd(X)
    qr_out = client.qr(X)
    pseudo = client.pinv(X)
    diagonal = client.diag([1, 2, 3])
    reshaped = client.reshape([[1, 2], [3, 4]], 4, 1)
    merged = client.concat([[1], [2]], [[3], [4]], axis=1)

    print("svd", svd_out)
    print("qr", qr_out)
    print("pinv", pseudo)
    print("diag", diagonal)
    print("reshape", reshaped)
    print("concat", merged)
```

Recommended testing order

1. Start with `health()` from `test_app.py` to verify connectivity.
2. Run `matmul`, `det`, and `rank` first because they are simple validation checks.
3. Run decompositions such as `eig`, `svd`, `qr`, and `cholesky` only after basic calls work.
4. Test `reshape` and `concat` last because they depend heavily on input shape assumptions.

Timeouts are best supplied on every outbound request when using `Requests`, because the library otherwise waits until the socket layer decides to fail, which can make debugging slower and less predictable.[cite:497][cite:495]

Common issues

- **Import error for config:** keep `config.py` in the same folder as `la_client.py`, because the client imports `get_base_url` from that local module.[cite:460]
- **ModuleNotFoundError: requests:** activate the virtual environment and rerun `pip install -r requirements.txt`.[cite:450]

- **Timeouts or connection failures:** verify network access, API key, and endpoint availability; Requests supports explicit timeout control for this reason.[cite:497][cite:499]
- **Shape or value errors:** the SDK validates that matrices are non-empty and rectangular before sending the request, so malformed inputs will fail early in Python before reaching the API.

Notes

The bundle is best used as a small local SDK project rather than a pip package. If needed later, the same client can be repackaged into a distributable module while preserving the current requests-based transport layer and internal base-URL config approach.[cite:497][cite:498]